

Frankfurt University
of Applied Sciences

– Fachbereich 2: Informatik und Ingenieurwissenschaften –

Deep-Learning-basierte Anomalieerkennung in Passantenfrequenzen

Projektarbeit im Rahmen des Bachelorstudiums

Bachelor of Science (B.Sc.)

vorgelegt am 23. Juli 2026 von

Sherwin Nourinfar, Wassim Hanane

Matrikelnummer: 1552887, 1583937

Referent	:	Daniel Helmer
Korreferent	:	–

Eidesstattliche Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die im Literaturverzeichnis angegebenen Quellen benutzt habe.

Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder noch nicht veröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht.

Die Zeichnungen oder Abbildungen in dieser Arbeit sind von mir selbst erstellt worden oder mit einem entsprechenden Quellennachweis versehen.

Diese Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht worden.

Frankfurt, 23. Juli 2026



Sherwin Nourinfar, Wassim Hanane

Inhaltsverzeichnis

1 Einführung **2**

1.1 Motivation . . . 2

1.2 Zielsetzung . . . 2

1.3 Aufbau der Arbeit . . . 2

2 Hintergrund **3**

3 Stand der Technik **4**

3.1 Prognosebasierte Anomalieerkennung . . . 4

3.2 Anomalieerkennung durch Rekonstruktion . . . 4

3.3 Komplexität und fairer Vergleich . . . 4

4 Entwickelte Methode **6**

4.1 Datengrundlage . . . 6

4.2 Datenqualität . . . 6

4.3 Bereinigung der Mehrfachmessungen . . . 7

4.4 Behandlung der Messlücken . . . 7

4.5 Kritische Betrachtung . . . 8

4.6 Forecast-basierter Anomaliedetektor . . . 10

4.6.1 Lernaufgabe und Eingaberepräsentation . . . 10

4.6.2 Vom Ursprungs-Forecaster zur aktuellen Architektur . . . 10

4.6.3 Nichtkonformitätsscore und Intervallkalibrierung . . . 12

4.6.4 Kausale Hybridregel . . . 12

4.6.5 Umgang mit imputierten Zielwerten . . . 13

4.7 Denoising-Autoencoder . . . 13

5 Evaluation der Methode **14**

5.1 Zeitliche Datentrennung . . . 14

5.2 Vergleich mit dem Ursprungs-Forecaster . . . 14

5.3 Bewertung der Prognosegüte . . . 15

5.4 Synthetische Anomalieinjektion . . . 15

5.5 Punkt- und Ereignismetriken . . . 15

5.6 Reproduzierbarkeit 16

6 Ergebnisse **17**

6.1 Training und Detektorkalibrierung 17

6.2 Prognosegüte im Testabschnitt 18

6.3 Kandidaten im unveränderten Datensatz 18

6.4 Ergebnisse der Anomalieinjektion 18

6.5 Vergleich der Modellstände 19

6.6 Diskussion und Limitationen 20

7 Zusammenfassung und Ausblick **21**

A Programmcode **22**

Literatur **25**

Kurzfassung

Kapitel 1

Einführung

Wie viele Menschen zu einer bestimmten Zeit an einem Ort unterwegs sind, folgt meist einem klaren Rhythmus. Tagsüber sind es mehr als nachts, und unter der Woche andere Zahlen als am Wochenende. Auch der Ort spielt eine Rolle, denn an einem belebten Platz ist mehr los als in einer ruhigen Seitenstraße. Wenn man dieses regelmäßige Verhalten kennt, fallen Abweichungen davon auf. Eine solche Abweichung vom gewohnten Muster nennt man Anomalie.

Grundlage dieser Arbeit ist ein realer Datensatz der Stadt Bad Nauheim. An 30 Standorten wurde vom 30. Juni bis zum 13. Juli 2025 stündlich gezählt, wie viele Passanten unterwegs waren. Für jeden Sensor entsteht so eine eigene Zeitreihe über zwei Wochen.

1.1 Motivation

Mit diesem Projekt wollen wir herausfinden, ob sich solche Anomalien mit Methoden des Deep Learnings zuverlässig erkennen lassen und welche Modellart sich dafür am besten eignet. Die Schwierigkeit dabei ist, dass die Daten nicht beschriftet sind. Es ist nirgends vermerkt, welche Stunde normal war und welche nicht. Ein Modell muss das normale Verhalten also selbst aus den Daten lernen und melden, wenn die Realität davon abweicht. Feste Schwellenwerte helfen hier kaum, weil sich das Normalverhalten je nach Tageszeit, Wochentag und Standort verändert.

1.2 Zielsetzung

Das Ziel ist ein System, das für jeden Sensor das normale Passantenaufkommen lernt und auffällige Stunden meldet. Dafür werden ein LSTM-Forecaster und ein LSTM-Autoencoder umgesetzt und anhand derselben künstlichen Anomalien und Kennzahlen verglichen. Ein Transformer bleibt wegen der kurzen Datenreihe eine mögliche spätere Erweiterung. So lässt sich begründen, welcher der tatsächlich getesteten Ansätze für diese Daten besser geeignet ist.

1.3 Aufbau der Arbeit

Kapitel 2 erklärt die nötigen Grundlagen zu Zeitreihen, neuronalen Netzen und den verwendeten Architekturen. Anschließend ordnet der Stand der Technik die Arbeit in bestehende Verfahren ein. Danach folgen die entwickelte Methode mit Datenaufbereitung und Training, die Evaluation der Modelle und die Ergebnisse. Den Abschluss bildet eine Zusammenfassung mit Ausblick.

Kapitel 2

Hintergrund

Kapitel 3

Stand der Technik

3.1 Prognosebasierte Anomalieerkennung

Hundman et al. verwenden LSTM-Prognosefehler und dynamische Schwellen zur Erkennung von Anomalien in Zeitreihen [5]. Globale rekurrente Prognosemodelle können Informationen über mehrere verwandte Zeitreihen teilen und zugleich reihenspezifische Merkmale berücksichtigen [12]. Quantilprognosen bilden zusätzlich die Unsicherheit einer Vorhersage ab, statt nur einen einzelnen Schätzwert auszugeben [3, 8]. Dropout kann die Anpassung eines Netzes an seine Trainingsbeispiele begrenzen [16].

Diese Arbeiten begründen die im Projekt verwendeten Bausteine Prognosefehler, gemeinsames Lernen, Sensorinformation, Quantilausgabe und Dropout. Sie geben jedoch keine optimale Schichtbreite für den kurzen Datensatz aus Bad Nauheim vor. Die konkreten Größen werden deshalb in Abschnitt 4.6.2 anhand von Datenumfang und Parameterbudget festgelegt.

3.2 Anomalieerkennung durch Rekonstruktion

Ein Autoencoder lernt, normale Eingabefolgen zu komprimieren und anschließend wiederherzustellen. Große Rekonstruktionsfehler dienen danach als Hinweis auf eine Abweichung. Malhotra et al. übertragen dieses Prinzip auf mehrdimensionale Sensordaten. Ihr Modell verwendet jeweils eine LSTM-Schicht im Encoder und im Decoder und zeigt, dass ein solcher Aufbau sowohl periodische als auch weniger gut vorhersagbare Reihen verarbeiten kann [10]. Die Arbeit begründet den verwendeten Grundaufbau, gibt aber keine allgemein optimale Zahl von LSTM-Einheiten für andere Datensätze vor.

Denoising-Autoencoder erhalten beim Training eine teilweise verdeckte Eingabe und sollen trotzdem die unveränderten Werte rekonstruieren. Dadurch wird das reine Kopieren der Eingabe erschwert [19]. Neuere Verfahren verwenden ebenfalls zeitliche Maskierung, kombinieren diese jedoch teilweise mit mehreren Transformern, Frequenzdarstellungen und zusätzlichen Fehlerfunktionen [2]. Für den kurzen Datensatz dieses Projekts wird nur das einfache Maskierungsprinzip übernommen.

3.3 Komplexität und fairer Vergleich

Aktuelle Vergleichsarbeiten warnen davor, eine komplexere Architektur automatisch als besseren Anomaliedetektor zu betrachten. Sarfraz et al. zeigen, dass zusätzliche Komplexität häufig nur geringe Vorteile bringt, und empfehlen einfache, verständliche Vergleichsmodelle [13]. Der umfangreiche TSB-AD-Vergleich kommt ebenfalls zu dem Ergebnis, dass einfache neuronale und statistische Verfahren viele moderne Netze übertreffen können [9].

Deshalb wird kein Anspruch erhoben, mit einer bestimmten Schichtgröße einen allgemein besten Stand der Technik zu erreichen. Stattdessen werden aktuelle Erkenntnisse auf einen kompakten Autoencoder übertragen und mit denselben Zeitabschnitten, künstlichen Anomalien und Kennzahlen wie der Forecaster geprüft. Eine zusätzliche Schicht wird nur

dann gerechtfertigt, wenn sie auf dieser gemeinsamen Prüfung einen nachvollziehbaren Vorteil liefert.

Kapitel 4

Entwickelte Methode

Dieses Kapitel beschreibt zunächst die reproduzierbare Aufbereitung der Rohdaten und anschließend den entwickelten forecast-basierten Anomaliedetektor. Methodische Entscheidungen werden von den später berichteten Ergebnissen getrennt dargestellt.

4.1 Datengrundlage

Die Rohdaten liegen in Form einer JSON-Datei mit 17.578 Einzelmessungen vor. Jede Messung gehört zu einem von 30 Sensorstandorten in der Innenstadt und im Kurpark von Bad Nauheim und enthält neben dem Zeitstempel die Anzahl der erfassten Passanten (`visitors`) sowie deren mittlere Verweildauer (`avgDuration`). Der globale Erfassungszeitraum reicht vom 30. Juni bis zum 13. Juli 2025 und umfasst 335 Stunden. Da nicht alle Sensoren während des gesamten Zeitraums aktiv waren, wird jede Reihe nur zwischen ihrer ersten und letzten Messung betrachtet. Dadurch entstehen 9.460 aktive Sensorstunden; weitere 590 Kombinationen des globalen Rasters liegen außerhalb der jeweiligen Sensorlaufzeit und werden nicht als Messlücke behandelt.

4.2 Datenqualität

Die explorative Analyse mit `analyse/analyse.py` untersucht zunächst Aufbau, Mehrfachmessungen, Verteilungen und Beziehungen der Messgrößen. Die anschließende Bildung eines vollständigen Stundenrasters in `analyse/preprocessing_plots.py` macht zusätzlich fehlende Stunden sichtbar. In vorhandenen Rohzeilen fehlen weder `visitors` noch `avgDuration`; eine ausgefallene Stunde fehlt jedoch als ganze Zeile und wäre mit einer reinen Suche nach leeren Einzelwerten nicht erkennbar.

Mehrfachmessungen: Für 4.770 Kombinationen aus Sensor und Stunde enthält der Datensatz mehrere Einträge, in der Spitze zehn Messungen für dieselbe Stunde (Abbildung 4.1). Die Werte dieser Einträge weichen teilweise voneinander ab, es handelt sich also nicht um exakte Kopien. Ohne Bereinigung gingen diese Stunden mehrfach und mit widersprüchlichen Werten in das Training ein.

Messlücken: Nach der Zusammenfassung der Mehrfachmessungen liegen 6.842 gemessene Sensorstunden vor. Innerhalb der 9.460 aktiven Sensorstunden fehlen damit 2.618 Werte, entsprechend 27,7 %. Sie bilden 830 zusammenhängende Lücken. Davon umfassen 641 höchstens zwei Stunden; die längste Lücke innerhalb einer aktiven Sensorperiode dauert 18 Stunden (Abbildung 4.2). In 62 Stunden liegt von keinem der zu diesem Zeitpunkt aktiven Sensoren eine Messung vor. Dies deutet auf gemeinsame Ausfälle der Übertragungsinfrastruktur hin. Die Sensoren Kurpark–Eingang Ludwigstraße und Friedrichstraße 1 besitzen mit 36 beziehungsweise 59 Stunden lediglich kurze aktive Erfassungszeiträume. Zeiten nach ihrem letzten Messwert werden nicht künstlich ergänzt.

Tabelle 4.1 fasst die wichtigsten Folgerungen der Analyse zusammen. Damit wird jede Aufbereitungsentscheidung aus einem beobachteten Datenmerkmal abgeleitet.

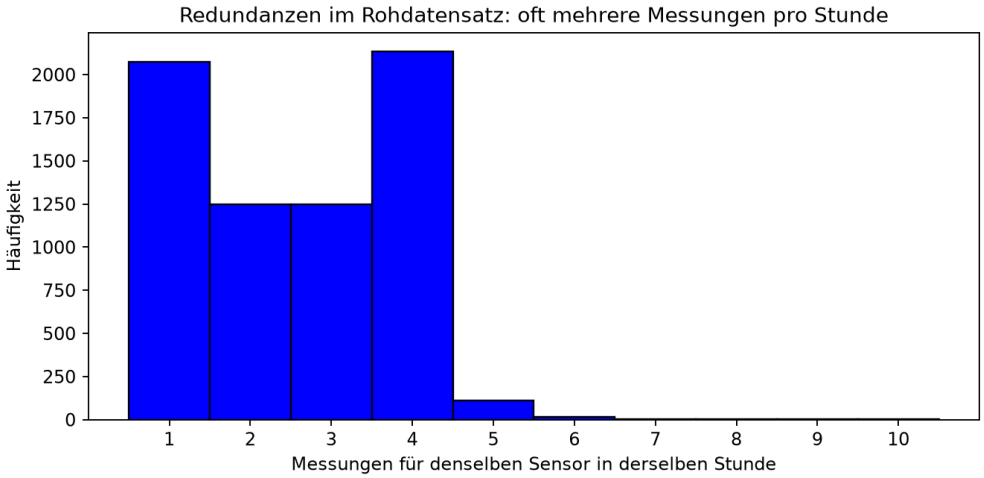


Abbildung 4.1: Häufigkeit von Mehrfachmessungen im Rohdatensatz. Für viele Stunden liegen zwei bis vier, vereinzelt bis zu zehn Einträge desselben Sensors vor.

Tabelle 4.1: Aus der Datenanalyse abgeleitete Aufbereitung. Eigene Darstellung.

Analysebefund	Folgerung für die Aufbereitung
Bis zu zehn widersprüchliche Einträge je Sensorstunde	Zusammenfassung zu genau einem Messpunkt je Stunde
2.618 vollständig fehlende Stunden innerhalb aktiver Sensorzeiten	Stundenraster, Imputation und dauerhafte Kennzeichnung der Ersatzwerte
641 von 830 Lücken sind höchstens zwei Stunden lang	vergangenheitsbasierte Fortschreibung auf zwei Stunden begrenzen
Tages- und Wochenrhythmus sowie stark unterschiedliche Sensorniveaus	Kalendermerkmale, Zeitprofile und sensorweise Skalierung verwenden
Mittlere sensorweise Korrelation von nur 0,13 zwischen den Zielgrößen	`visitors` und `avgDuration` getrennt beibehalten

4.3 Bereinigung der Mehrfachmessungen

Alle Einträge derselben Kombination aus Sensor und Stunde werden zu einem Messpunkt zusammengefasst. Für die Passantenzahl und die Verweildauer wird jeweils der arithmetische Mittelwert gebildet, da keine der mehrfachen Messungen als verlässlichere Quelle gekennzeichnet ist. Dadurch verbleiben 6.842 eindeutige gemessene Sensorstunden. Nach Abschluss der Imputation wird die Passantenzahl auf ganze Personen und die Verweildauer auf zwei Dezimalstellen gerundet.

4.4 Behandlung der Messlücken

Die für das Training verwendeten Zeitfenster setzen zusammenhängende Stundenreihen voraus. Eine Lücke innerhalb eines Fensters würde andernfalls unbemerkt Werte aus weit auseinanderliegenden Zeiträumen kombinieren. Die Lücken werden deshalb vor dem Training aufgefüllt, und zwar für jeden Sensor getrennt, damit jeder Standort sein individuelles Niveau und Tagesprofil behält. Die zentrale Pipeline verwendet keine lineare Interpolation. Sämtliche Profile werden ausschließlich aus ursprünglich gemessenen Werten vor dem 8. Juli 2025, 00:00 Uhr UTC, geschätzt; Validierungs-, Kalibrierungs- und Testwerte gehen damit nicht in die Profilbildung ein. Wochentag und Stunde werden in der lokalen Zeitzone `Europe/Berlin` bestimmt. Das Verfahren arbeitet in vier aufeinanderfolgenden Stufen.

Stufe 1, vergangenheitsbasierte Fortschreibung: Da 641 der 830 Lücken und damit 77,2 % höchstens zwei Stunden dauern, wird die Fortschreibung auf genau diesen Zeitraum begrenzt. Für die ersten zwei Stunden jeder Lücke wird der

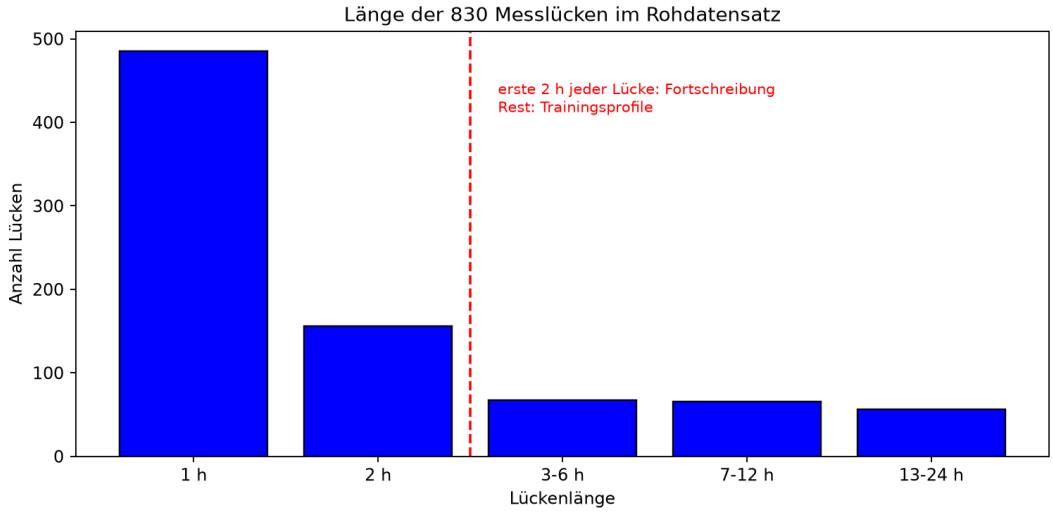


Abbildung 4.2: Längenverteilung der 830 Messlücken innerhalb der aktiven Sensorperioden. 641 Lücken umfassen höchstens zwei Stunden.

letzte zuvor gemessene Wert desselben Sensors übernommen. Mit t^- als Zeitpunkt der letzten Messung gilt

$$\hat{x}_t = x_{t^-}, \qquad 0 < t - t^- \leq 2 \text{ Stunden}. \tag{4.1}$$

Diese Regel gilt auch für die ersten beiden Positionen einer längeren Lücke. Sie benötigt keinen nachfolgenden Messwert und verhindert dadurch, dass ein späteres Prognoseziel in einen früheren Modellkontext zurückgespiegelt wird. Der Preis ist eine kurzfristig konstante Fortsetzung; die Regel stellt daher einen transparenten Kompromiss und kein allgemein optimales Imputationsverfahren dar. Auf diese Weise werden 1.175 Sensorstunden ergänzt.

Stufe 2, Wochentag-Stunden-Profil: Für die verbleibenden Lücken wird der Mittelwert desselben Sensors am gleichen lokalen Wochentag und zur gleichen lokalen Stunde aus gemessenen Trainingswerten eingesetzt. Historische Zeitprofile erhalten tages- und wochenzeitabhängige Strukturen und werden auch für fehlende Verkehrszähldaten untersucht [17]. Diese Stufe füllt 1.298 Sensorstunden.

Stufe 3, Stundenprofil: Existiert für eine Kombination aus Wochentag und Stunde keine Trainingsreferenz, wird der Mittelwert des Sensors zur gleichen lokalen Uhrzeit über alle Trainingstage verwendet. Dies betrifft 140 Sensorstunden.

Stufe 4, Sensormittel: Fehlt auch für die betreffende Uhrzeit eine Referenz, wird der Mittelwert sämtlicher gemessener Trainingswerte dieses Sensors eingesetzt. Diese Rückfallstufe betrifft fünf Sensorstunden. Damit werden alle 2.618 Lücken innerhalb der aktiven Sensorperioden geschlossen; Zeiten vor der ersten und nach der letzten Messung eines Sensors bleiben außerhalb des Datensatzes.

Der untere Teil von Abbildung 4.3 kennzeichnet die ergänzten Stunden. Abbildung 4.4 stellt eine typische Reihe und den Sensor mit der kürzesten aktiven Erfassungszeit gegenüber. Sie zeigt zugleich, dass die Reihe nach dem Ende des Sensors nicht künstlich fortgeschrieben wird. Abbildung 4.5 fasst den ergänzten Anteil innerhalb der aktiven Zeit jedes Sensors zusammen.

4.5 Kritische Betrachtung der Bereinigung

Jede Imputation verändert die Daten. Mittelwertbasierte Verfahren können den Verlauf glätten und die Varianz verringern [20, 15]. Im vorliegenden Datensatz ändert sich der Mittelwert der Passantenzahl je Sensor um höchstens 9,92 %. Die kleinste Korrelation zwischen dem ausschließlich aus gemessenen Werten und dem aus allen bereinigten Werten gebildeten stündlichen Tagesprofil beträgt 0,9838. Diese Kennzahlen prüfen Niveau und Profil, ersetzen aber keine Fehleranalyse gegen unbekannte wahre Werte in den Lücken.

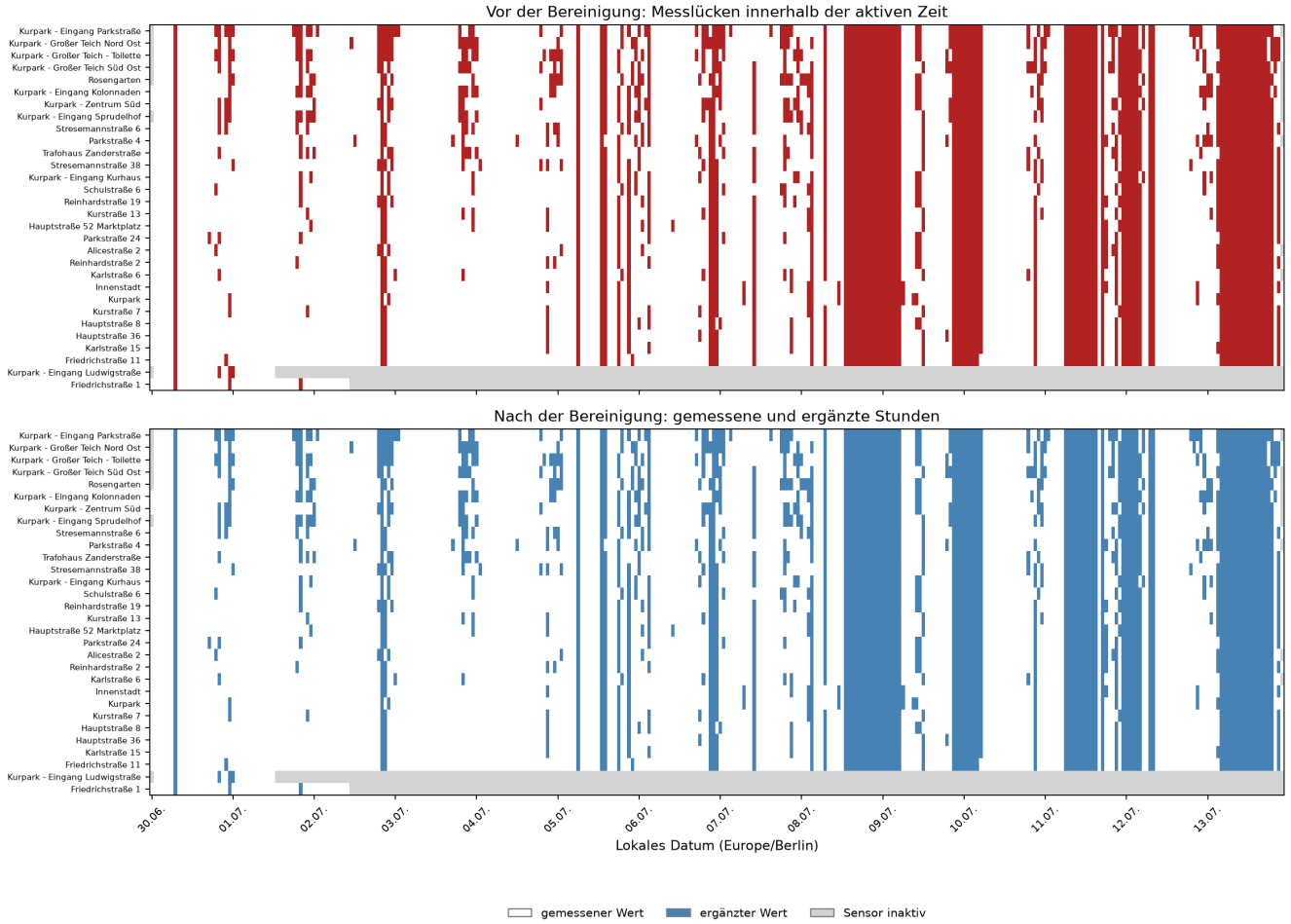


Abbildung 4.3: Messstatus vor der Bereinigung (oben) und danach (unten). Rot zeigt Lücken innerhalb der aktiven Sensorzeit, Blau ergänzte Werte und Grau Zeiträume, in denen der betreffende Sensor nicht aktiv war. Eigene Darstellung.

Die zweistündige Fortschreibung verwendet zu jedem Zeitpunkt nur die Vergangenheit. Die statistischen Profile dürfen dagegen innerhalb des Trainingsabschnitts auch chronologisch spätere Trainingsmessungen enthalten. Die Aufbereitung ist damit für die Out-of-Sample-Auswertung frei von Validierungs- und Testdaten-Leakage, jedoch nicht als rein online-kausale Imputation jeder frühen Trainingslücke zu verstehen.

Die erste und letzte vorhandene Messung dienen mangels eines Betriebsplans nur als vorläufige Grenzen der Sensoraktivität. War ein Sensor planmäßig schon früher oder noch länger aktiv, wären die dort fehlenden Stunden als Ausfälle zu behandeln. Die vorgesehenen Aktivitätszeiträume müssen deshalb noch mit den Betreiberdaten abgeglichen werden.

Ein zweiter Aspekt betrifft die Anomalieerkennung selbst. Ergänzte Stunden wirken per Konstruktion unauffällig, ein tatsächlicher Sensorausfall stellt jedoch selbst ein meldenswertes Ereignis dar. Das Fehlen von Werten folgt hier zudem eigenen, teils blockweisen Mustern und trägt damit selbst Information [20]. Das Feld `is_imputed` hält für jede Stunde fest, ob sie gemessen oder ergänzt wurde. Der Forecaster verwendet ergänzte Werte weiterhin als Kontext eines 24-Stunden-Fensters, überspringt jedoch Fenster mit einem ergänzten Zielwert beim Training, bei der Bestimmung des Schwellenwerts und bei der Testauswertung. Die beiden besonders kurzen Sensorreihen werden wegen ihrer geringen aktiven Länge nicht verwendet.

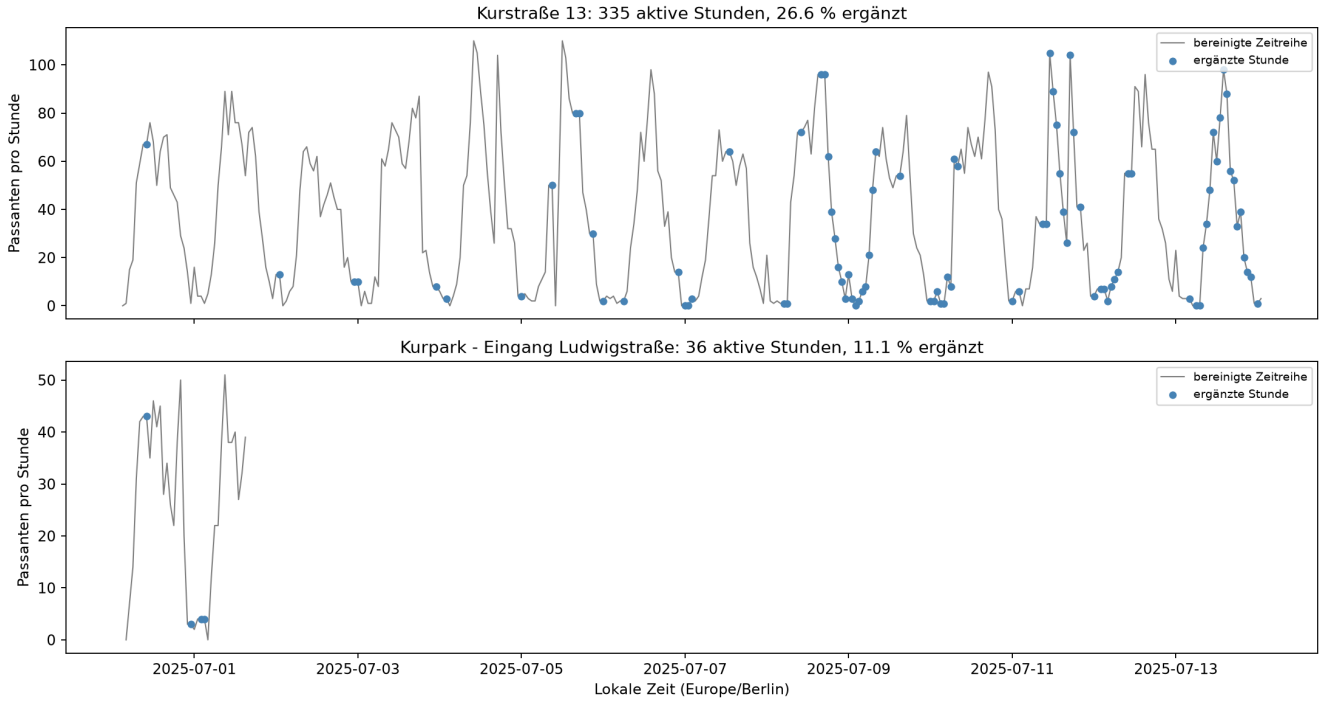


Abbildung 4.4: Bereinigte Zeitreihen eines typischen Sensors (oben) und des Sensors mit der kürzesten aktiven Erfassungszeit (unten). Markierte Punkte sind ergänzte Stunden. Eigene Darstellung.

4.6 Forecast-basierter Anomaliedetektor

4.6.1 Lernaufgabe und Eingaberepräsentation

Der Forecaster lernt selbstüberwacht, aus den jeweils vorhergehenden 24 Stunden die nächste Stunde vorherzusagen. Die nachgelagerte Anomalieentscheidung ist unüberwacht, da weder das Modelltraining noch die Kalibrierung reale Anomalieklassen verwenden. Ein einziges globales Modell wird über 28 Sensoren trainiert. Zwei Sensoren werden ausgeschlossen, weil vor dem Ende des Trainingsabschnitts weniger als 72 gemessene Zielstunden vorliegen.

Beide Zielgrößen werden zunächst mit $\log(1+x)$ transformiert. Danach erfolgt eine robuste Skalierung je Sensor s und Merkmal f:

$$z_{s,t,f} = \frac{\log(1+x_{s,t,f}) - \mathrm{Median}_{s,f}}{\max(\mathrm{IQR}_{s,f}, 0{,}05)}. \tag{4.2}$$

Median und Interquartilsabstand werden ausschließlich aus gemessenen Trainingswerten bestimmt. Die sensorweisen Mittelwerte der Passantenzahl reichen in der Analyse von etwa 9,1 bis 262,5 Personen pro Stunde; die sensorweise Skalierung verhindert, dass diese Niveauunterschiede die Modellierung dominieren. Jedes historische Zeitfenster besitzt acht Kanäle: die beiden skalierten Messgrößen, eine Maske für gemessen beziehungsweise imputiert sowie Sinus und Kosinus von lokaler Stunde und Wochentag und einen Wochenendindikator. Die fünf Kalendermerkmale der Zielstunde und eine Sensor-ID werden dem Modell zusätzlich übergeben.

4.6.2 Vom Ursprungs-Forecaster zur aktuellen Architektur

Der erste Forecaster des Projekts bestand aus

$$\mathrm{Input}(24,2) \rightarrow \mathrm{LSTM}(64) \rightarrow \mathrm{Dense}(2). \tag{4.3}$$

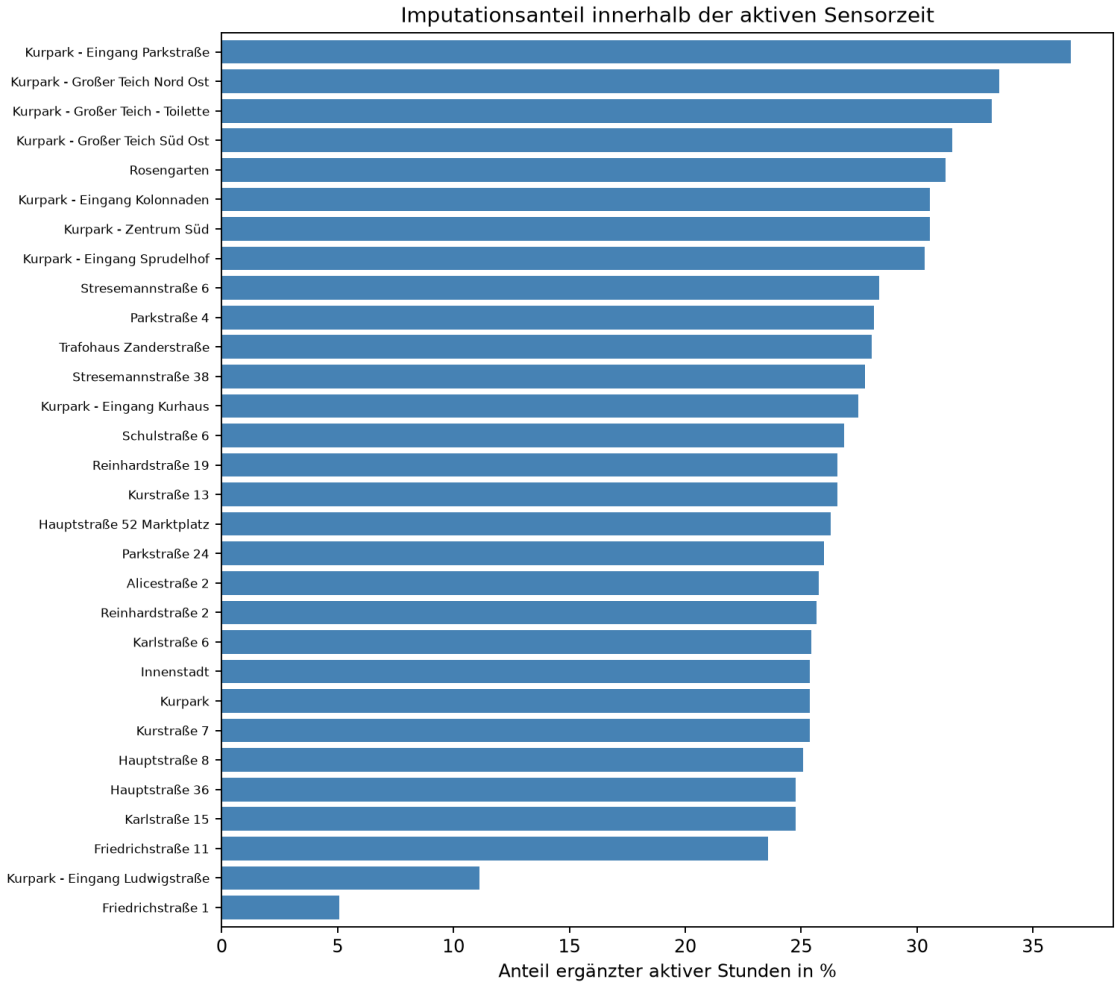


Abbildung 4.5: Anteil ergänzter Stunden innerhalb der aktiven Zeit jedes Sensors. Eigene Darstellung.

Er besaß 17.282 trainierbare Parameter und erzeugte mit MSE trainierte Punktprognosen für beide Messgrößen. Dieser verständliche Ausgangspunkt berücksichtigte jedoch weder Prognoseunsicherheit noch Sensoridentität oder bekannte Kalendereigenschaften der Zielstunde.

Aus den Literaturbefunden in Abschnitt 3.1 werden daher Quantilausgaben, ein kleines Sensor-Embedding, Kalendermerkmale und Dropout übernommen. Der einfache Kern aus einer einzelnen LSTM-Schicht bleibt erhalten. Tabelle 4.2 zeigt die Änderung einmalig; der faire Ergebnisvergleich folgt in Abschnitt 6.5.

Das LSTM verdichtet das 24-Stunden-Fenster in einen Zustandsvektor [4]. Dieser wird mit einem lernbaren Sensor-Embedding und den Kalendermerkmalen der Zielstunde verbunden und durch eine dichte Schicht geführt. Das globale Training und das Sensor-Embedding orientieren sich an probabilistischen globalen RNN-Forecastern [12]; aufgrund der Quantilausgabe handelt es sich jedoch nicht um eine DeepAR-Implementierung.

Das Ausgabefeld enthält für `visitors` und `avgDuration` jeweils die Quantile $q_{0,1}$, $q_{0,5}$ und $q_{0,9}$. Trainiert wird mit der über beide Zielgrößen und Quantile gemittelten Pinball-Loss-Funktion [8]. Die konkrete Breite des Netzes wird nicht aus einem fremden Datensatz übernommen, sondern am Umfang der eigenen Daten und an einem kleinen Parameterbudget ausgerichtet. Tabelle 4.3 zeigt diese Festlegung.

Insgesamt besitzt das Modell 22.798 trainierbare Parameter. Die Größen 64, 48 und 6 bilden gemeinsam eine kompakte Projektkonfiguration; sie sind keine einzeln aus der Literatur übernommenen Optimalwerte. Breitere oder gestapelte LSTM-Schichten werden wegen des kurzen Messzeitraums nicht ergänzt. Die endgültige Architektur wird 21 Epochen

Tabelle 4.2: Ursprungs-Forecaster und aktuelle Architektur. Eigene Darstellung.

Aspekt	Ursprungs-Forecaster	Aktueller Forecaster
Kontext	zwei Messgrößen	Messgrößen, Beobachtungsmaske und Kalender
Sensorbezug	keiner	gemeinsames Modell mit Sensor-Embedding
Ausgabe	zwei Punktprognosen	drei Quantile je Messgröße
Training	MSE	Pinball Loss und Dropout
Anomalieentscheidung	absoluter Prognosefehler	kalibriertes Intervall und kausale Persistenz

Tabelle 4.3: Größenwahl des Forecasters. Eigene Darstellung.

Baustein	Größe	Parameter	Begründung für diesen Datensatz
LSTM	64	18.688	Verdichtet ein Fenster aus 24 Stunden und acht Kanälen. Wegen nur 4.111 gemessener Trainingsziele bleibt es bei einer rekurrenten Schicht.
Sensor-Embedding	6	168	Bildet 28 Sensoren mit $28 \cdot 6$ Werten als kleine Zusatzinformation ab.
Dense	48	3.648	Mischt den 75-dimensionalen Vektor aus LSTM, Embedding und fünf Kalenderwerten und verkleinert ihn auf 48 Werte.
Dropout	0,25	0	Begrenzt die Anpassung an den kurzen und durch überlappende Fenster stark ähnlichen Trainingszeitraum.
Ausgabe	6	294	Durch zwei Messgrößen und drei Quantile je Messgröße vollständig vorgegeben.

auf Training und Tuning trainiert; Kalibrierungs- und Testdaten bleiben dabei unberührt.

4.6.3 Nichtkonformitätsscore und Intervallkalibrierung

Für Merkmal f wird die Abweichung einer Beobachtung $y_{t,f}$ vom vorhergesagten Quantilintervall relativ zu dessen Breite bewertet:

$$e_{t,f} = \frac{\max\{q_{0,1,t,f} - y_{t,f},\, y_{t,f} - q_{0,9,t,f}\}}{\max\{q_{0,9,t,f} - q_{0,1,t,f},\, w_{\min,f}\}}, \qquad s_t = \max_f e_{t,f}. \tag{4.4}$$

Ein negativer Wert liegt innerhalb des ursprünglichen Quantilintervalls; ein positiver Wert misst die relative Überschreitung. Das Maximum s_t erzeugt eine gemeinsame Entscheidung für beide Zielgrößen. Aus 368 gemessenen Punkten des getrennten Kalibrierungsabschnitts wird bei nominal $\alpha = 0{,}01$ eine endliche empirische Quantilschwelle bestimmt. Das Verfahren ist an Conformalized Quantile Regression angelehnt [11], verwendet aber zusätzlich eine Breitenstandardisierung und ein Maximum über zwei Zielgrößen. Wegen der zeitlichen Abhängigkeit der Fenster wird keine verteilungsfreie 99-%-Garantie beansprucht [1]; die tatsächliche Testabdeckung wird berichtet.

4.6.4 Kausale Hybridregel

Starke Einzelabweichungen werden unmittelbar erkannt. Für moderate, aber anhaltende Abweichungen wird zusätzlich eine kausale Persistenzregel verwendet:

$$A_t = \mathbf{1}(s_t > \tau_{\text{hoch}}) \;\vee\; \mathbf{1}\left(\sum_{j=0}^{m-1} \mathbf{1}(s_{t-j} > \tau_{\text{niedrig}}) \geq k\right). \tag{4.5}$$

Die Regel arbeitet je Sensor, betrachtet nur aktuelle und vergangene Scores und wird bei einer Zeitlücke oder einem imputierten Ziel zurückgesetzt. Aus 18 vorab festgelegten Kombinationen werden τ_{hoch}, τ_{niedrig}, m und k ausschließlich anhand synthetischer Anomalien im Kalibrierungsabschnitt gewählt. Zulässig sind nur Regeln mit höchstens 1 % Alarmen auf den unveränderten Kalibrierungswerten. Optimiert wird der über die Anomalietypen gleichgewichtete Ereignis-F1; die Injektionen des Testabschnitts beeinflussen diese Auswahl nicht.

4.6.5 Umgang mit imputierten Zielwerten

Imputierte Werte dürfen als markierter Kontext in ein Zeitfenster eingehen, werden aber nicht als Ziel für Training, Modellauswahl, Kalibrierung oder Forecast-Metriken verwendet. Eine fehlende Originalmessung wird unabhängig vom Forecast als Datenqualitätsereignis ausgegeben. Dadurch wird ein aus einem Zeitprofil erzeugter Ersatzwert weder als reguläre Beobachtung noch als bestätigte Verhaltensanomalie fehlinterpretiert.

4.7 Denoising-Autoencoder

Der Autoencoder bildet eine zweite, vom Prognosemodell getrennte Methode. Er erhält ein 24-Stunden-Fenster mit den beiden skalierten Messgrößen und der Beobachtungsmaske. Seine Ausgabe rekonstruiert die beiden Messgrößen für alle 24 Stunden. Der vollständige Aufbau lautet:

$$\mathrm{Input}(24, 3) \to \mathrm{LSTM}(16) \to \mathrm{RepeatVector}(24) \to \mathrm{LSTM}(16) \to \mathrm{Dense}(2). \tag{4.6}$$

Die einzelne LSTM-Schicht auf beiden Seiten des komprimierten Zustands folgt dem Grundaufbau von Malhotra et al. [10]. Die Literatur bestimmt damit die Form, während Tabelle 4.4 die an den vorliegenden Datensatz angepassten Größen begründet.

Tabelle 4.4: Größenwahl des Autoencoders. Eigene Darstellung.

Baustein	Größe	Parameter	Begründung für diesen Datensatz
Eingabe	24×3	0	Ein Tag mit zwei Messgrößen und einer Beobachtungsmaske.
Encoder-LSTM	16	1.280	Verdichtet 48 Messwerte auf 16 Zustandswerte und bildet damit einen Engpass im Verhältnis 3:1.
RepeatVector	24	0	Stellt wieder genau die ursprüngliche Länge von 24 Stunden bereit.
Decoder-LSTM	16	2.112	Verwendet dieselbe Breite wie der Encoder und führt dadurch keine weitere frei gewählte Größe ein.
Dense	2	34	Durch die Rekonstruktion von `visitors` und `avgDuration` vollständig vorgegeben.

Das Netz besitzt insgesamt 3.426 trainierbare Parameter. Acht Zustandswerte wären für Tagesform, Niveau und das Verhältnis beider Messgrößen ein sehr enger Engpass. 32 oder mehr Werte würden die Verdichtung deutlich abschwächen und könnten ungewöhnliche Eingaben zu leicht rekonstruieren. Deshalb wird 16 als kleine mittlere Breite verwendet. Eine zusätzliche versteckte Dense-Schicht ist nicht nötig, da die vorhandene Rekonstruktion bereits beide Messgrößen direkt aus dem Decoderzustand erzeugt.

Während des Trainings wird in jedem Fenster ein zusammenhängender Block von einer bis vier wirklich gemessenen Stunden verdeckt. Damit bleiben selbst im größten Fall 20 der 24 Stunden als zeitlicher Kontext erhalten. Die zugehörigen Eingabewerte und ihre Maske werden auf null gesetzt, das Rekonstruktionsziel bleibt jedoch unverändert. Diese einfache Datenerweiterung folgt dem Denoising-Prinzip [19]. Künstliche Spikes, Dips oder andere Testanomalien werden nicht für das Training verwendet.

Trainiert wird mit Adam, Batchgröße 128, höchstens 100 Epochen und Early Stopping. Die Huber-Fehlerfunktion verringert den Einfluss einzelner großer, im unbeschrifteten Training möglicherweise bereits vorhandener Abweichungen. Ergänzte Werte dürfen als markierter Kontext eingehen, besitzen im Rekonstruktionsfehler aber das Gewicht null. Das Modell lernt sie daher nicht als echte Zielwerte.

Für jede Stunde wird der absolute Rekonstruktionsfehler der letzten Position des Fensters verwendet. Damit die beiden Messgrößen vergleichbar werden, wird jeder Fehler durch sein 75-%-Quantil im Kalibrierungsabschnitt geteilt; der größere der beiden Werte bildet den gemeinsamen Anomaliewert. Die Alarmregel wird anschließend mit demselben Kalibrierungsverfahren wie beim Forecaster festgelegt. Dadurch unterscheiden sich die Modelle, während Datenabschnitte, Injektionen und Entscheidungskriterien gleich bleiben.

Kapitel 5

Evaluation der Methode

5.1 Zeitliche Datentrennung

Alle Teilmengen werden chronologisch und nicht zufällig gebildet. Dies verhindert, dass spätere Stunden als Lernbeispiele für frühere Prognosen verwendet werden. Tabelle 5.1 nennt die Grenzen und die Anzahl tatsächlich gemessener Zielpunkte nach dem 24-Stunden-Windowing.

Tabelle 5.1: Chronologische Teilmengen beider Modelle. Alle Grenzen sind in UTC angegeben. Eigene Darstellung.

Teilmenge	Beginn	Ende (exklusiv)	Zielpunkte
Training	erster möglicher Zielwert	08.07.2025	4.111
Tuning	08.07.2025	09.07.2025	318
Kalibrierung	09.07.2025	10.07.2025	368
Test	10.07.2025	Reihenende	1.346

Die Forecaster-Architektur und ihre 21 Epochen sind vor dem endgültigen Lauf festgelegt. Für diesen Lauf werden Training und Tuning gemeinsam zum Anpassen der Gewichte verwendet. Beim Autoencoder dient der Tuningabschnitt dem Early Stopping. Anschließend bestimmt der Kalibrierungsabschnitt für beide Modelle die Schwellen und die Persistenzregel. Der Testabschnitt wird für keine dieser Entscheidungen verwendet.

5.2 Vergleich mit dem Ursprungs-Forecaster

Für den Vorher-Nachher-Vergleich wird nicht der historische Trainingslauf des Ursprungs-Forecasters verwendet, weil dieser noch auf den nicht zusammengefassten Rohzeilen arbeitete. Stattdessen wird seine ursprüngliche Architektur aus Abschnitt 4.6.2 auf derselben bereinigten Datenbasis wie der aktuelle Forecaster neu trainiert. Beide Ansätze verwenden dieselben Zeitgrenzen, gemessenen Zielstunden, Zufallsstartwerte, 21 Trainingsepochen und künstlichen Anomalieereignisse. Die absoluten Fehler des Ursprungsmodells werden je Zielgröße durch ihr 99-%-Quantil auf den unveränderten Kalibrierungswerten geteilt und anschließend durch das Maximum beider Zielgrößen zusammengeführt. Schwellen und Persistenzregel werden für beide Modellstände ausschließlich im Kalibrierungsabschnitt bestimmt; das Alarmbudget beträgt jeweils höchstens 1 % auf den unveränderten Kalibrierungswerten.

Der Vergleich bewertet damit die beiden vollständigen Modellstände unter gleichen Bedingungen. Er zeigt nicht den isolierten Anteil einer einzelnen Ergänzung wie Sensor-Embedding, Quantilausgabe oder Dropout. Eine solche Trennung würde mehrere zusätzliche Zwischenmodelle erfordern und ist für die Fragestellung dieses Projekts nicht notwendig.

5.3 Bewertung der Prognosegüte

Die Medianprognose $q_{0,5}$ wird mit MAE und RMSE bewertet. Zusätzlich wird die saisonal skalierte mittlere absolute Abweichung (MASE) nach Hyndman und Koehler berichtet [6]:

$$\mathrm{MASE} = \frac{H^{-1}\sum_{h=1}^{H}|y_{T+h} - \hat{y}_{T+h}|}{(T-24)^{-1}\sum_{t=25}^{T}|y_t - y_{t-24}|}. \tag{5.1}$$

Der Nenner wird ausschließlich aus gemessenen Trainingszielen berechnet. Damit ist MASE von dem ergänzend angegebenen Verhältnis $\mathrm{MAE}_{\mathrm{Modell,Test}} \,/\, \mathrm{MAE}_{t-24,\mathrm{Test}}$ zu unterscheiden. Werte unter eins bedeuten jeweils eine Verbesserung gegenüber der zugehörigen Skalierung beziehungsweise Testbaseline.

Für die empirisch kalibrierten Intervalle werden die marginale Abdeckung je Zielgröße, die gemeinsame Abdeckung beider Zielgrößen und die mittlere Intervallbreite ausgewiesen. Eine hohe Abdeckung allein ist nicht hinreichend, da sie durch sehr breite und damit wenig trennscharfe Intervalle erreichbar wäre.

5.4 Synthetische Anomalieinjektion

Reale Anomalieklassen liegen nicht vor. Deshalb wird die technische Erkennungsfähigkeit ergänzend mit kontrollierten Injektionen geprüft. Dieses Vorgehen liefert eindeutige Labels für genau definierte Fehlertypen, ersetzt aber keine Evaluation realer Ereignisse [14]. Die Anomalien werden ausschließlich in ursprünglich gemessene Werte injiziert, bevor Zeitfenster erzeugt und Prognosen berechnet werden. Damit beeinflussen mehrstündige Ereignisse den nachfolgenden Modellkontext wie reale Messwerte. Forecaster und Autoencoder erhalten dieselben ausgewählten Sensoren, Zeitpunkte, Merkmale, Richtungen und Stärken.

Untersucht werden ein einstündiger positiver Spike, ein einstündiger Dip, ein vierstündiger Level-Shift, ein über vier Stunden ansteigender Drift und ein sechsstündiger Stuck-at-Fehler. Mit Ausnahme von Stuck-at werden beide Zielgrößen und bei den mehrstündigen Typen beide Richtungen abgedeckt. Die Stärken 0,5, 1,0 und 2,0 beziehen sich auf sensor- und merkmalsabhängige robuste $\log(1+x)$-IQR-Einheiten. Stuck-at friert ein Merkmal auf die letzte vorherige Messung ein; Ereignisse mit weniger als 0,5 realisierter IQR-Abweichung werden verworfen, weil sie keine belastbare künstliche Veränderung darstellen.

Die Auswahl der Alarmregel verwendet drei feste Zufallsstartwerte im Kalibrierungsabschnitt und nur die Stärken 1,0 und 2,0. Der endgültige Vergleichstest verwendet drei andere feste Zufallsstartwerte im Testabschnitt und schließt 0,5 als Sensitivitätsanalyse ein. Pro Lauf wird höchstens ein Ereignis je Sensor erzeugt. Jede injizierte Reihe wird mit einem identischen unveränderten Gegenlauf gepaart. Ein Treffer zählt nur, wenn die Injektion einen zusätzlichen Alarm hervorruft; ein bereits am gleichen Zeitpunkt bestehender Alarm wird nicht nachträglich als Erfolg der Injektion gewertet.

5.5 Punkt- und Ereignismetriken

Auf Punktebene werden Konfusionsmatrix, Precision, Recall, F1, ROC-AUC, PR-AUC und zeitliche Intersection over Union berechnet. Precision und Recall sind durch

$$\mathrm{Precision} = \frac{TP}{TP+FP}, \qquad \mathrm{Recall} = \frac{TP}{TP+FN} \tag{5.2}$$

definiert. Die PR-AUC ist wegen des starken Ungleichgewichts zwischen Normal- und Anomaliepunkten besonders informativ; ROC-AUC wird ergänzend ausgegeben.

Zusammenhängende stündliche Alarme desselben Sensors werden zusätzlich als Ereignisse gruppiert. Berichtet werden Ereignis-Precision, Ereignis-Recall, Ereignis-F1, mittlere zeitliche IoU und die Verzögerung bis zum ersten überlappenden Alarm. Diese Existenz- und Überlappungsdefinition ist eine bewusst einfache ereignisorientierte Auswertung und nicht das vollständige Metriksystem von Tatbul et al. [18]. Die ursprünglichen Punktvorhersagen werden nicht verändert. Insbesondere wird kein Point Adjustment verwendet, da es die gemessene Leistung stark überschätzen kann [7].

5.6 Reproduzierbarkeit

Die explorative Ausgangsanalyse ist in `analyse/analyse.py` erhalten; `analyse/preprocessing_plots.py` erzeugt die Vorher-Nachher-Abbildungen der Aufbereitung. Die eigentliche Pipeline wird aus dem Projektverzeichnis mit folgenden Befehlen ausgeführt:

```
py -3.12 -m src.preprocessing
py -3.12 -m src.forecaster.train
py -3.12 -m src.autoencoder.train
py -3.12 -m src.benchmark
py -3.12 -m src.forecaster.evaluate
```

Bei unveränderten Rohdaten werden nur die gewünschten Modelle neu trainiert. Für die erneute Auswertung des gespeicherten Forecasters reicht `py -3.12 -m src.forecaster.evaluate`. Der davon getrennte synthetische Vergleich beider Modelle wird mit `py -3.12 -m src.benchmark` ausgeführt. Modellparameter, Splits, Skalierung, Kalibrierung und Detektorregel werden in `model_bundle.json` gespeichert. Die Auswertung erzeugt unter anderem `evaluation_metrics.json`, `anomalies.csv`, `anomaly_events.csv` und die Abbildung `anomalies_example.png`. Der gemeinsame Injektionstest schreibt die Detailergebnisse je Modell und fasst beide zusätzlich in `saved_models/benchmark/model_comparison.csv` zusammen.

Kapitel 6

Ergebnisse

6.1 Training und Detektorkalibrierung

Die in Abschnitt 4.6 begründete feste Architektur wird 21 Epochen auf 4.429 gemessenen Zielen aus Training und Tuning trainiert. Der getrennte Kalibrierungsabschnitt und der Testabschnitt werden dabei nicht zum Anpassen der Modellgewichte verwendet. Der Trainingsverlauf ist in Abbildung 6.1 dargestellt.

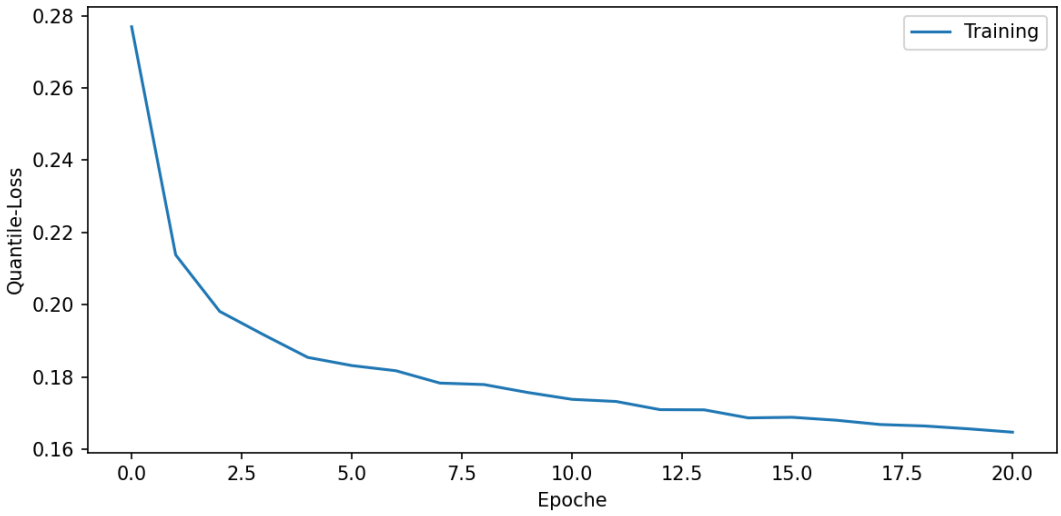


Abbildung 6.1: Pinball-Loss des endgültigen Forecaster-Trainings. Eigene Darstellung.

Die gemeinsame empirische 1-%-Schwelle beträgt 0,7486. Unter den 15 zulässigen Alarmregeln wählt die Kalibrierung zusätzlich die Bedingung, dass mindestens drei der letzten vier lückenlosen Messscores den niedrigeren Wert 0,3082 überschreiten. Auf den unveränderten 368 Kalibrierungspunkten erzeugt diese Regel zwei Alarme beziehungsweise 0,54 % und hält damit das vorab festgelegte 1-%-Budget ein. Der Macro-Ereignis-F1 der Kalibrierungsinjektionen beträgt 0,746. Dieser Wert ist ein Auswahlkriterium und kein Testergebnis.

Tabelle 6.1: Prognosegüte auf 1.346 gemessenen Testzielen. Die Abdeckung und Breite beziehen sich auf die empirisch kalibrierten Intervalle. Eigene Darstellung.

Ziel	MAE	RMSE	MASE	rel. MAE	Abdeckung	Breite
Visitors	12,114	22,962	0,622	0,745	99,63 %	224,99
AvgDuration	4,124	7,540	0,855	0,821	99,55 %	98,54

6.2 Prognosegüte im Testabschnitt

Beide MASE-Werte und beide relativen Test-MAE-Werte liegen unter eins. Die Medianprognose ist damit im Mittel genauer als die jeweils verwendete saisonale Skalierung beziehungsweise $t-24$-Testbaseline. Die gemeinsame empirische Abdeckung beider Zielgrößen beträgt 99,26 %. Gleichzeitig sind die Intervalle insbesondere für die Passantenzahl breit. Die hohe Abdeckung ist deshalb zusammen mit der Breite zu interpretieren und belegt allein keine hohe Trennschärfe des Anomaliedetektors.

6.3 Kandidaten im unveränderten Datensatz

Die eingefrorene Hybridregel markiert 15 der 1.346 gemessenen Testpunkte. Sie bilden elf zusammenhängende Forecast-Ereignisse. Die ursprüngliche reine 1-%-Punktschwelle hätte zehn Punkte markiert; die Persistenzregel ergänzt somit fünf moderate Punkte. Abbildung 6.2 zeigt den Sensor mit den meisten Forecast-Kandidaten. Die rote Markierung bedeutet eine statistische Abweichung und keine fachlich bestätigte Anomalie.

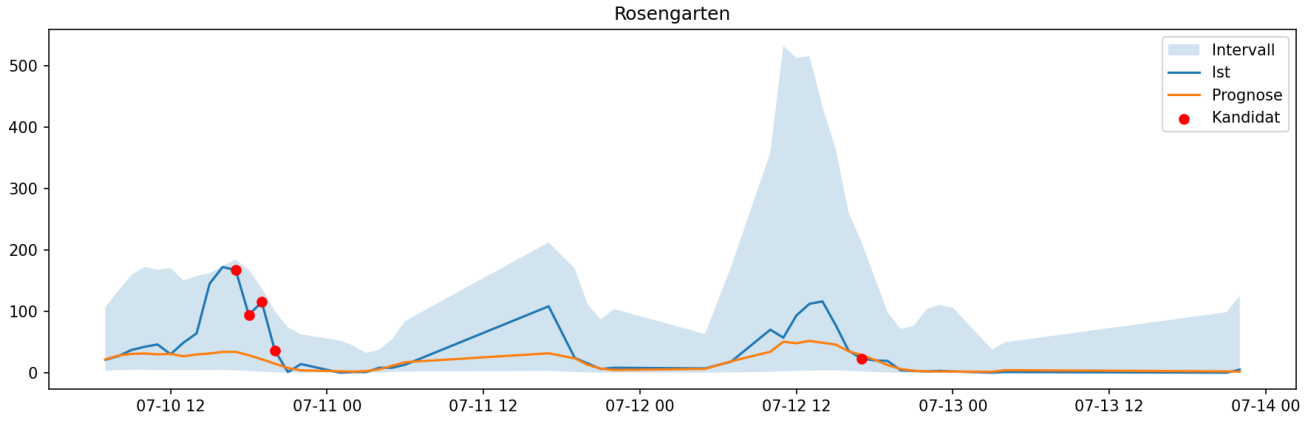


Abbildung 6.2: Messung, Medianprognose, empirisch kalibriertes Intervall und Forecast-Kandidaten eines Beispielsensors. Eigene Darstellung.

Zusätzlich besitzen 1.333 Testziele keine Originalmessung. Diese werden zu 275 zusammenhängenden Datenlücken-Ereignissen gruppiert und separat ausgewiesen. Sie sind Hinweise auf fehlende Sensordaten, aber nicht automatisch Anomalien des Passantenverhaltens. Werden Forecast- und Datenqualitätsalarme gemeinsam gruppiert, entstehen 282 Ereignisse; diese Zahl darf daher nicht als Anzahl erkannter Verhaltensanomalien interpretiert werden.

6.4 Ergebnisse der Anomalieinjektion

Tabelle 6.2 fasst die starken Injektionen der Stärke 2,0 und den gefilterten sechs Stunden langen Stuck-at-Test über drei Testseeds zusammen. Die Referenzspalten zeigen die unveränderte reine 1-%-Punktschwelle. Sämtliche Treffer sind inkrementell gegenüber dem jeweils gepaarten unveränderten Lauf.

Tabelle 6.2: Punkt- und Ereignisergebnisse der Testinjektionen. R = Recall, Ref. = Referenzdetektor. Eigene Darstellung.

Typ	Punkt-R	Punkt-F1	Ereignis-R	Ereignis-F1	Ref.-R	Ref.-F1
Spike, 2,0	0,633	0,732	0,633	0,767	0,633	0,775
Dip, 2,0	0,533	0,651	0,533	0,658	0,533	0,672
Level-Shift, 2,0	0,708	0,742	0,861	0,878	0,833	0,883
Drift, 2,0	0,458	0,543	0,750	0,810	0,694	0,780
Stuck-at, 6 h	0,237	0,345	0,371	0,499	0,315	0,404

Die Persistenzregel erhöht den Ereignis-Recall erwartungsgemäß bei mehrstündigen Veränderungen: bei starkem Level-Shift von 0,833 auf 0,861, bei starkem Drift von 0,694 auf 0,750 und bei Stuck-at von 0,315 auf 0,371. Für einstündige Spikes und Dips entsteht kein Recall-Gewinn. Bei Level-Shift und den punktuellen Typen sinkt der Ereignis-F1 geringfügig, weil zusätzliche Alarme die Precision belasten. Die Regel verbessert somit nicht jede einzelne Kennzahl, erhöht aber gezielt die Sensitivität für anhaltende Abweichungen, ohne die empirische Kalibrierungs-Alarmrate zu erhöhen.

Schwache Injektionen bleiben schwierig. Bei Stärke 0,5 beträgt der Ereignis-Recall 0,067 für Spike, 0,017 für Dip, 0,167 für Level-Shift und 0,028 für Drift. Der Detektor erkennt folglich nicht jede definierte Anomalie. Die vollständigen Werte aller Stärken einschließlich Precision, PR-AUC, IoU und Verzögerung stehen in `injection_benchmark.csv` und `injection_benchmark.json`.

6.5 Vergleich der Modellstände

Tabelle 6.3 vergleicht den fair neu trainierten Ursprungs-Forecaster, den aktuellen Forecaster und den Autoencoder auf denselben starken Testinjektionen. Dazu werden die fünf Anomaliearten gleich gewichtet gemittelt. Die künstlichen Ereignisse und die Alarmregel des Testabschnitts wurden für kein Modell zur Auswahl der Netzschichten verwendet.

Tabelle 6.3: Gemeinsamer Vergleich auf starken künstlichen Anomalien. Eigene Darstellung.

Modell	Parameter	Punkt-F1	PR-AUC	Ereignis-R	Ereignis-F1
Ursprungs-Forecaster	17.282	0,393	0,250	0,486	0,535
Forecaster	22.798	0,603	0,502	0,630	0,722
Autoencoder	3.426	0,425	0,237	0,552	0,581

Gegenüber dem Ursprungs-Forecaster steigt der Punkt-F1 des aktuellen Systems von 0,393 auf 0,603, die PR-AUC von 0,250 auf 0,502, der Ereignis-Recall von 0,486 auf 0,630 und der Ereignis-F1 von 0,535 auf 0,722. Relativ entspricht dies Verbesserungen von 53,3 %, 101,0 %, 29,5 % und 34,9 %. Der Vergleich belegt den Unterschied zwischen den beiden vollständigen Modellständen; er weist nicht nach, welchen Einzelanteil Sensor-Embedding, Quantilausgabe, Dense-Schicht oder Dropout daran haben.

Die Verbesserung betrifft vor allem die Anomalietrennung und nicht jede Prognosekennzahl. Für `visitors` sinkt die MAE von 12,596 auf 12,114 und die MASE von 0,647 auf 0,622. Bei `avgDuration` liegt das Ursprungsmodell mit einer MAE von 4,108 und einer MASE von 0,851 knapp vor den aktuellen Werten 4,124 und 0,855. Ein allgemein kleinerer Prognosefehler allein erklärt die bessere Anomalieerkennung somit nicht.

Der Autoencoder benötigt rund 85 % weniger Parameter, erkennt die geprüften Anomalien aber insgesamt schlechter. Seine höchsten Ereignis-F1-Werte erreicht er bei Level-Shifts mit 0,764 und Drifts mit 0,698. Gerade für diese mehrstündigen Veränderungen liefert die Rekonstruktion somit ein brauchbares zusätzliches Ergebnis. Der Forecaster liegt jedoch bei jeder der fünf Anomaliearten vorn und bleibt deshalb für den aktuellen Datensatz das stärkere Einzelmodell. Eine Mischung beider Anomaliewerte wurde nicht vorgenommen, damit Prognose und Rekonstruktion getrennt beurteilt werden können.

6.6 Diskussion und Limitationen

Die Ergebnisse beantworten die Forschungsfrage differenziert. Der globale Quantil-Forecaster übertrifft die saisonale Baseline im mittleren Prognosefehler. Starke punktuelle und mehrstündige Injektionen werden teilweise bis überwiegend erkannt; die zusätzliche Persistenz verbessert vor allem Level-Shift, Drift und Stuck-at. Für schwache Anomalien und einzelne Dips ist die Sensitivität gering. Der kompakte Autoencoder bestätigt, dass Rekonstruktionsfehler grundsätzlich nutzbar sind, erreicht auf diesen Daten aber nicht die Erkennungsleistung des Forecasters.

Die synthetische Ground Truth gilt ausschließlich für die definierten Transformationen. Sie belegt weder den Recall unbekannter realer Anomalien noch die fachliche Richtigkeit der elf Kandidaten im Originaldatensatz. Der nur zwei Wochen umfassende Datensatz begrenzt außerdem die beobachtbaren Wochenmuster und die Größe von Tuning und Kalibrierung. Zeitliche Abhängigkeit verhindert eine klassische verteilungsfreie Conformal-Garantie. Schließlich wurde der Testzeitraum im Verlauf der retrospektiven Entwicklung bereits betrachtet; die Testinjektionen sind durch neue Zufallsstartwerte und eine festgelegte Regel getrennt, stellen aber keine vollständig unangetastete konfirmatorische Studie dar.

Kapitel 7

Zusammenfassung und Ausblick

Die aufbereiteten Sensordaten wurden mit zwei getrennten Verfahren untersucht: Ein Forecaster bewertet Abweichungen von seiner Prognose, ein kompakter Autoencoder bewertet Rekonstruktionsfehler. Beide Modelle verwenden dieselben Zeitabschnitte, ursprünglich gemessenen Zielwerte und künstlichen Testanomalien. Dadurch ist der Vergleich nachvollziehbar, ohne die beiden Anomaliewerte miteinander zu vermischen.

Der Forecaster erkennt die geprüften Anomaliearten insgesamt besser. Der Autoencoder ist deutlich kleiner und reagiert besonders auf mehrstündige Verschiebungen und Drifts, erreicht aber nicht die gleiche Zuverlässigkeit. Mehr Schichten sind mit den kurzen Datenreihen daher derzeit nicht begründet. Als nächste Schritte sind vor allem die manuelle Prüfung realer Kandidaten, längere Messreihen und zeitlich wiederholte Tests sinnvoll. Erst mit deutlich mehr Daten sollte zusätzlich ein Transformer untersucht werden.

Anhang A

Programmcode

Literatur

[1] Rina Foygel Barber u. a. "Conformal Prediction Beyond Exchangeability". In: *The Annals of Statistics* 51.2 (2023), S. 816–845. DOI: 10.1214/23-AOS2276.

[2] Yuchen Fang u. a. "Temporal-Frequency Masked Autoencoders for Time Series Anomaly Detection". In: *2024 IEEE 40th International Conference on Data Engineering*. 2024, S. 1228–1241. DOI: 10.1109/ICDE60146.2024.00099.

[3] Jan Gasthaus u. a. "Probabilistic Forecasting with Spline Quantile Function RNNs". In: *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*. Bd. 89. Proceedings of Machine Learning Research. 2019, S. 1901–1910. URL: https://proceedings.mlr.press/v89/gasthaus19a.html.

[4] Sepp Hochreiter und Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), S. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.

[5] Kyle Hundman u. a. "Detecting Spacecraft Anomalies Using LSTMs and Nonparametric Dynamic Thresholding". In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2018, S. 387–395. DOI: 10.1145/3219819.3219845.

[6] Rob J. Hyndman und Anne B. Koehler. "Another Look at Measures of Forecast Accuracy". In: *International Journal of Forecasting* 22.4 (2006), S. 679–688. DOI: 10.1016/j.ijforecast.2006.03.001.

[7] Siwon Kim u. a. "Towards a Rigorous Evaluation of Time-Series Anomaly Detection". In: *Proceedings of the AAAI Conference on Artificial Intelligence* 36.7 (2022), S. 7194–7201. DOI: 10.1609/aaai.v36i7.20680.

[8] Roger Koenker und Jr. Bassett Gilbert. "Regression Quantiles". In: *Econometrica* 46.1 (1978), S. 33–50. DOI: 10.2307/1913643.

[9] Qinghua Liu und John Paparrizos. "The Elephant in the Room: Towards a Reliable Time-Series Anomaly Detection Benchmark". In: *Advances in Neural Information Processing Systems*. Bd. 37. 2024. DOI: 10.52202/079017-3437.

[10] Pankaj Malhotra u. a. "LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection". In: *ICML 2016 Anomaly Detection Workshop*. 2016. DOI: 10.48550/arXiv.1607.00148.

[11] Yaniv Romano, Evan Patterson und Emmanuel J. Candès. "Conformalized Quantile Regression". In: *Advances in Neural Information Processing Systems*. Bd. 32. 2019, S. 3543–3553. URL: https://proceedings.neurips.cc/paper_files/paper/2019/hash/5103c3584b063c431bd1268e9b5e76fb-Abstract.html.

[12] David Salinas u. a. "DeepAR: Probabilistic Forecasting with Autoregressive Recurrent Networks". In: *International Journal of Forecasting* 36.3 (2020), S. 1181–1191. DOI: 10.1016/j.ijforecast.2019.07.001.

[13] M. Saquib Sarfraz u. a. "Position: Quo Vadis, Unsupervised Time Series Anomaly Detection?" In: *Proceedings of the 41st International Conference on Machine Learning*. Bd. 235. Proceedings of Machine Learning Research. 2024, S. 43461–43476. URL: https://proceedings.mlr.press/v235/sarfraz24a.html.

[14] Sebastian Schmidl, Phillip Wenig und Thorsten Papenbrock. "Anomaly Detection in Time Series: A Comprehensive Evaluation". In: *Proceedings of the VLDB Endowment* 15.9 (2022), S. 1779–1797. DOI: 10.14778/3538598.3538602.

[15] Muhammad Awais Shafique. "Imputing Missing Data in Hourly Traffic Counts". In: *Sensors* 22.24 (2022), S. 9876. DOI: 10.3390/s22249876.

[16] Nitish Srivastava u. a. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting". In: *Journal of Machine Learning Research* 15.56 (2014), S. 1929–1958. URL: https://www.jmlr.org/papers/v15/srivastava14a.html.

[17] Tuo Sun u. a. "Traffic Missing Data Imputation: A Selective Overview of Temporal Theories and Algorithms". In: *Mathematics* 10.14 (2022), S. 2544. DOI: 10.3390/math10142544.

[18] Nesime Tatbul u. a. "Precision and Recall for Time Series". In: *Advances in Neural Information Processing Systems*. Bd. 31. 2018, S. 1920–1930. URL: https://proceedings.neurips.cc/paper_files/paper/2018/hash/8f468c873a32bb0619eaeb2050ba45d1-Abstract.html.

[19] Pascal Vincent u. a. "Extracting and Composing Robust Features with Denoising Autoencoders". In: *Proceedings of the 25th International Conference on Machine Learning*. 2008, S. 1096–1103. DOI: 10.1145/1390156.1390294.

[20] Yimei Zhang u. a. "A Comprehensive Survey on Traffic Missing Data Imputation". In: *IEEE Transactions on Intelligent Transportation Systems* 25.12 (2024), S. 19252–19275. DOI: 10.1109/TITS.2024.3478816.